

Table 1: Configured model tiers and intended roles. This is a design table, not a performance or cost result.

Tier	Model	Intended role
Fast	GPT-4.1-mini	Classification and simple extraction
Standard	GPT-4.1	Primary reasoning and code generation
Strong	GPT-4.1	Reflection and refinement

provider-token boundary, because provider tokenizers and image accounting vary. Provider-reported usage remains authoritative. Frozen benchmark drivers retain the base observational `LLMClient`; their batch usage is reconciled from immutable journals, terminal counters, and result artifacts.

4 Spatial Scene Graph Engine

The spatial scene graph engine is the cornerstone of the spatial question-answering path and the unexecuted FieldWorkArena adapter. It is designed to make spatial computations explicit, but it does not remove errors introduced by perception, depth estimation, object matching, or coordinate assumptions.

4.1 Problem Formulation

Given an image I of an industrial environment (factory, warehouse, or retail space) and a natural language question q , the task is to produce an answer a that may require counting objects, estimating distances, checking spatial containment, or verifying safety compliance. Directly prompting a VLM with (I, q) is unreliable because VLMs hallucinate spatial relationships and struggle with precise counting.

4.2 Scene Graph Construction

Our approach decomposes the problem into three stages: *extraction*, *structuring*, and *computation*.

Stage 1: Entity Extraction: We employ a two-pass extraction process. First, a vision-language model (GPT-4.1 with vision) generates a detailed textual description of the scene, prompted to enumerate all visible objects with approximate positions and attributes. Second, Florence-2 [23], a lightweight vision foundation model, performs object detection to obtain precise bounding boxes and counts, serving as a grounding mechanism for the VLM’s descriptions.

Stage 2: Graph Construction: Extracted entities are formalized as a spatial scene graph $G = (V, E)$ where vertices V represent entities and edges E represent spatial relations:

$$v_i = \text{SpatialEntity}(\text{id}_i, \text{label}_i, \text{pos}_i, \text{attrs}_i, \text{zone}_i) \quad (1)$$

$$e_{ij} = \text{SpatialRelation}(\text{subj}_i, \text{pred}_{ij}, \text{obj}_j, d_{ij}) \quad (2)$$

where $\text{pos}_i \in \mathbb{R}^2$ denotes the estimated position (from bounding box centroids), attrs_i is a dictionary of visual attributes (color, size, state), zone_i identifies the semantic zone (e.g., loading dock, aisle 3), and d_{ij} is the computed Euclidean distance between entities.

Stage 3: Deterministic Computation: The scene graph supports several query operations that produce verifiable facts:

- `query_near( $v, r$ )`: Returns all entities within radius r of entity v .
- `check_constraints( $C$ )`: Evaluates a set of spatial constraints C (e.g., minimum clearance distances) and returns violations.
- `count_by_label( $\ell$ )`: Returns the count of entities matching label ℓ , cross-referenced with Florence-2 detections.
- `to_fact_sheet()`: Serializes the graph into a structured natural language summary suitable for LLM consumption.

The fact sheet is then provided to the LLM alongside the original question, enabling it to answer based on computed facts rather than visual estimation.

Scoring Functions: The local FieldWorkArena adapter implements six scoring interfaces based on the available task specification. Because the gated benchmark data were not accessible, these interfaces have not been validated against an official FieldWorkArena evaluation run and are retained as dormant compatibility code.